

Programación 3

Emparejamiento estable

1. Defina el problema del **emparejamiento estable** y explique qué significa que un emparejamiento tenga una inestabilidad.
2. Demuestre que el algoritmo de Gale-Shapley siempre termina después de un número finito de pasos. ¿Cuál es la cota máxima de propuestas?
3. Explique por qué el resultado con hombres proponiendo es óptimo para ellos y pesimista para las mujeres.
4. Suponga que un hospital tiene varios cupos (generalización). Explique cómo se adapta el algoritmo y qué propiedad se mantiene.
5. Muestre un ejemplo en el que cambiar quién propone (hombres/mujeres) cambia el emparejamiento resultante.

DFS y BFS

6. Diseñe un algoritmo con BFS para determinar si un grafo no dirigido es bipartito.
7. Explique cómo usar DFS para detectar ciclos en un grafo dirigido.
8. Proponga un algoritmo $O(m+n)$ que, dado un nodo s , devuelva todos los nodos alcanzables desde s .
9. Diseñe un algoritmo basado en DFS que determine si un grafo no dirigido es conexo.
10. ¿Cómo puede usarse BFS para encontrar el **camino más corto en cantidad de aristas** desde un nodo fuente en un grafo no ponderado?

DAG y orden topológico

11. Explique cómo obtener un orden topológico de un DAG usando DFS.
12. Diseñe un algoritmo que, dado un DAG, determine si existe **único orden topológico**. Justifique.

13. Construya un ejemplo de un DAG donde existan múltiples órdenes topológicos.
14. Explique cómo usar un orden topológico para calcular el **camino más largo** en un DAG.

Greedy

15. Explique el algoritmo greedy de **Interval Scheduling** (orden por tiempos de finalización) y justifique su corrección.
16. Dé un contraejemplo mostrando que elegir intervalos por tiempo de inicio no es óptimo.
17. Formule el problema de **Interval Partitioning** y muestre por qué el número de recursos usados es al menos d , donde d es la mayor superposición.
18. Muestre un contraejemplo donde ordenar intervalos por tiempo de finalización en **Interval Partitioning** no da una solución óptima.
19. Formule el problema de **Scheduling to Minimize Lateness** y explique el algoritmo correcto.
20. Justifique la corrección de ese algoritmo usando un argumento de intercambio.
21. Defina el problema de **Hitting Points** y explique cómo resolverlo greedy.
22. Dé un ejemplo donde ordenar los intervalos por longitud creciente no resuelve Hitting Points.

Árboles de expansión mínima (MST)

23. Defina y demuestre la **propiedad de corte**.
24. Defina y demuestre la **propiedad de ciclo**.
25. Ejecute **Prim** paso a paso en un grafo pequeño de ejemplo.
26. Ejecute **Kruskal** en el mismo grafo y compare.
27. Explique por qué Prim y Kruskal siempre producen un árbol de mínimo costo.
28. Dé un ejemplo donde haya más de un MST (cuando hay pesos repetidos).

Dijkstra

29. Explique en qué se basa la elección del vértice agregado en cada iteración de Dijkstra.

30. Ejecute Dijkstra paso a paso en un grafo de 6 vértices con pesos no negativos.
31. Explique qué propiedad asegura que, una vez que un nodo entra en el conjunto S , su distancia no mejora.
32. Dé un ejemplo de grafo con aristas negativas donde Dijkstra da un resultado incorrecto.